

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODU 14 – SORTING



NAMA : Fauzi Ulhaq Abdillah

NIM : 109082500141

KELAS : S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori Modul 14

- IdeAlgoritmaSelection Sort
- Algoritma Selection Sort
- Ide AlgoritmaInsertion Sort
- AlgoritmaInsertionSort

B. Guided Modul 14

1.

```
2. package main
3.
4. import "fmt"
5.
6. func SelectionSortArray(arr *[8]int) {
7.     var idx_min, i, j int
8.     for i = 0; i < len(arr)-1; i++ {
9.         idx_min = i
10.        for j = i + 1; j < len(arr); j++ {
11.            if arr[j] < arr[idx_min] {
12.                idx_min = j
13.            }
14.        }
15.        if idx_min != i {
16.            arr[i], arr[idx_min] = arr[idx_min], arr[i]
17.        }
18.    }
19.}
20.
21. func main() {
22.     var arrAngka [8]int
23.
24.     for i := 0; i < len(arrAngka); i++ {
25.         fmt.Printf("Masukkan data angka indeks ke-%d : ", i)
26.         fmt.Scan(&arrAngka[i])
27.     }
28.     fmt.Println()
29.
30.     fmt.Println("=== SEBELUM SORTING ===")
31.     for i := 0; i < len(arrAngka); i++ {
32.         fmt.Print(arrAngka[i], " ")
33.     }
34.     fmt.Println()
35.
36.     SelectionSortArray(&arrAngka)
```

```

37.     fmt.Println("=== SETELAH SORTING ===")
38.     for i := 0; i < len(arrAngka); i++ {
39.         fmt.Print(arrAngka[i], " ")
40.     }
41.}
42.
43.

```

Output :

The screenshot shows a Go IDE with a file named `guided1.go`. The code implements a Selection Sort algorithm. The `main` function initializes an array `arrAngka` with 8 elements: 45, 12, 89, 7, 23, 5, 60, 11. It then calls `SelectionSortArray` to sort the array. The output in the terminal shows the array before and after sorting.

```

package main

import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idx_min, i, j int
    for i = 0; i < len(arr)-1; i++ {
        idx_min = i
        for j = i + 1; j < len(arr); j++ {
            if arr[j] < arr[idx_min] {
                idx_min = j
            }
        }
        if idx_min != i {
            arr[i], arr[idx_min] = arr[idx_min], arr[i]
        }
    }
}

func main() {
    var arrAngka [8]int
    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka indeks ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }

    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(arrAngka); i++ {

```

```

PS C:\Users\Hp\Documents\109082500141_F
auzi-Ulhaq-Abdillah\Week 12 - Modul 14>
go run guided1.go
Masukkan data angka indeks ke-0 : 45
Masukkan data angka indeks ke-1 : 12
Masukkan data angka indeks ke-2 : 89
Masukkan data angka indeks ke-3 : 7
Masukkan data angka indeks ke-4 : 23
Masukkan data angka indeks ke-5 : 5
Masukkan data angka indeks ke-6 : 60
Masukkan data angka indeks ke-7 : 11

=== SEBELUM SORTING ===
45
12
89
7
23
5
60
11

=== SETELAH SORTING ===
5
11
7
12
23
45
60
89
PS C:\Users\Hp\Documents\109082500141_F
auzi-Ulhaq-Abdillah\Week 12 - Modul 14>

```

2.

```

package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ {
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ {

```

```

        if sMHS[j].IPK < sMHS[idx_min].IPK {
            idx_min = j
        }
    }
    if idx_min != i {
        sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa ke-%d ---\n", i)

        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM : ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Printf(" === SEBELUM SORTING === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)
    fmt.Println(" === SETELAH SRORTING === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

}

Output :

The image shows a Windows IDE (VS Code) with a Go program for sorting a slice of student data. The code is written in Go and uses the 'fmt' package for input/output. The program defines a struct 'mahasiswa' with fields 'nama', 'nim', and 'ipk'. It implements a 'SelectionSortArray' function that sorts a slice of 'mahasiswa' structs by their 'ipk' field. The 'main' function reads 5 students' data and sorts them by IPK.

```
1  import "fmt"
2
3  type mahasiswa struct {
4      nama, nim string
5      ipk      float64
6  }
7
8  func SelectionSortArray(sMHS *[5]mahasiswa) {
9
10     var idx_min, i, j int
11     for i = 0; i < len(sMHS)-1; i++ {
12         idx_min = i
13         for j = i + 1; j < len(sMHS); j++ {
14             if sMHS[j].IPK < sMHS[idx_min].IPK {
15                 idx_min = j
16             }
17         }
18         if idx_min != i {
19             sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
20         }
21     }
22 }
23
24 func main() {
25     var structMHS [5]mahasiswa
26
27     for i := 0; i < len(structMHS); i++ {
28         fmt.Printf("--- Masukkan Data Mahasiswa ke-%d ---\n", i)
29         fmt.Print("Nama : ")
30         fmt.Scan(&structMHS[i].nama)
31         fmt.Print("NIM : ")
32         fmt.Scan(&structMHS[i].nim)
33     }
```

The terminal output shows the program's execution, displaying the input data and the sorted output:

```
PS C:\Users\Hp\Documents\109082500141_Fauzi-Ulhaq-Abdillah\Week 12 - Modul 14>
go run guided2.go
--- Masukkan Data Mahasiswa ke-0 ---
Nama : Fauzi
NIM : 109082500141
IPK : 3.20
--- Masukkan Data Mahasiswa ke-1 ---
Nama : Persib
NIM : 109082500153
IPK : 3.85
--- Masukkan Data Mahasiswa ke-2 ---
Nama : Amuh
NIM : 10908250043
IPK : 3.50
--- Masukkan Data Mahasiswa ke-3 ---
Nama : Endang
NIM : 109082500231
IPK : 2.95
--- Masukkan Data Mahasiswa ke-4 ---
Nama : Tohari
NIM : 109082500423
IPK : 3.70

=== SEBELUM SORTING === --- Data Mahas
iswa ke-0 ---
Nama : Fauzi
NIM : 109082500141
IPK : 3.20
--- Data Mahasiswa ke-1 ---
Nama : Persib
NIM : 109082500153
IPK : 3.85
--- Data Mahasiswa ke-2 ---
Nama : Amuh
NIM : 10908250043
```

3.

```
package main

import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i
        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan data: ")

    for i = 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

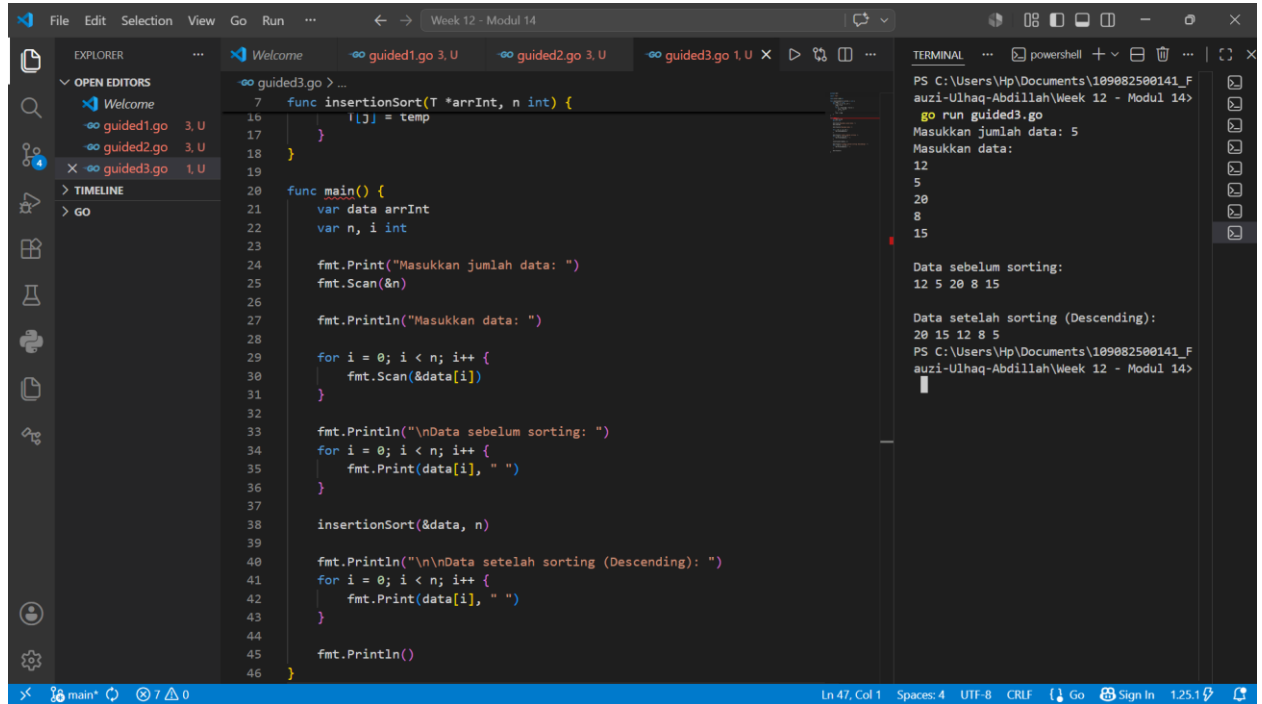
    fmt.Println("\nData sebelum sorting: ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)

    fmt.Println("\n\nData setelah sorting (Descending): ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}
```

```
    fmt.Println()
}
```

Output :



4.

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    ipk  float64
}

type arrMHS [100]mahasiswa

func insertionSortStruct(T *arrMHS, n int) {
    var temp mahasiswa
    var i, j int

    for i = 1; i < n; i++ {
        temp = T[i]
        j = i - 1
        for j >= 0 && T[j].nama < temp.nama {
            T[j+1] = T[j]
            j = j - 1
        }
        T[j+1] = temp
    }
}

func main() {
    var data arrMHS
    var n, i int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Println("\nData mahasiswa ke-", i+1)

        fmt.Print("Nama : ")
        fmt.Scan(&data[i].nama)

        fmt.Print("NIM : ")
    }
}
```



```

        fmt.Scan(&data[i].nim)

        fmt.Print("IPK : ")
        fmt.Scan(&data[i].ipk)
    }

    fmt.Printf("\nData sebelum sorting:\n")
    for i = 0; i < n; i++ {
        // KOREKSI: Menggunakan Println agar formatnya benar
        dan tidak eror
        fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
    }

    insertionSortStruct(&data, n)

    fmt.Println("\nData setelah sorting (Descending
    berdasarkan nama): ")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
    }
}

```

Output :

The screenshot shows a Go IDE with the following components:

- EXPLORER:** Lists several files including `guided1.go`, `guided2.go`, `guided3.go`, and `guided4.go`.
- EDITOR:** Displays the source code for `guided4.go`. The code defines a `mahasiswa` struct with fields `nama` (string), `nim` (string), and `ipk` (float64). It includes an `insertionSortStruct` function and a `main` function that prompts the user for the number of data entries.
- TERMINAL:** Shows the program's output. It displays data for four students (Amuh, Fauzi, Asep, Sobirin) before and after sorting. The output after sorting is in descending order based on the `nama` field.

Output from the terminal:

```

Data mahasiswa ke- 1
Nama : Amuh
NIM : 109082500123
IPK : 3.50

Data mahasiswa ke- 2
Nama : Fauzi
NIM : 109082500141
IPK : 3.75

Data mahasiswa ke- 3
Nama : Asep
NIM : 109082500043
IPK : 3.80

Data mahasiswa ke- 4
Nama : Sobirin
NIM : 109082500125
IPK : 3.60

Data sebelum sorting:
Amuh 109082500123 3.5
Fauzi 109082500141 3.75
Asep 109082500043 3.8
Sobirin 109082500125 3.6

Data setelah sorting (Descending berdas
arkan nama):
Sobirin 109082500125 3.6
Fauzi 109082500141 3.75
Asep 109082500043 3.8
Amuh 109082500123 3.5
PS C:\Users\Hp\Documents\109082500141_F
auzi-Ulhaq-Abdillah\Week 12 - Modul 14>

```

C. Unguided Modul 14

1. Soal Unguided 1 – Selection Sort

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi baris berikutnya selalu dimulai dengan sebuah integer ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar dimasing masing daerah.

Jawaban :

```
package main

import "fmt"

func selectionSortAsc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)
```

```

        rumah := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        selectionSortAsc(rumah)

        for j := 0; j < m; j++ {
            fmt.Print(rumah[j])
            if j < m-1 {
                fmt.Print(" ")
            }
        }
        fmt.Println()
    }
}

```

Output :

The screenshot shows an IDE with a Go file named 'unguided1.go'. The code implements a selection sort algorithm. The output in the terminal window is as follows:

```

PS C:\Users\Hp\Documents\109082500141_F
auzi-Ulhaq-Abdillah\Week 12 - Modul 14>
go run unguided1.go
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9

```

Penjelasan :

Kode Go-Lang diatas itu untuk membaca jumlah daerah, lalu di setiap daerah membaca sejumlah nomor rumah milik kerabat Hercules. Program ini membaca beberapa kumpulan angka, mengurutkannya menggunakan Selection Sort Ascending, lalu menampilkan hasil pengurutan dari nilai terkecil ke terbesar.

2. Soal Unguided 2 – Selection Sort

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, dimasing-masing daerah.

Jawaban :

```
package main

import (
    "fmt"
    "sort"
)

func main() {
    var n int
    if _, err := fmt.Scan(&n); err != nil {
        return
    }

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            var nomor int
            fmt.Scan(&nomor)

            if nomor%2 != 0 {
                ganjil = append(ganjil, nomor)
            } else {
                genap = append(genap, nomor)
            }
        }
    }
```

```

sort.Ints(ganjil)

sort.Slice(genap, func(x, y int) bool {
    return genap[x] > genap[y]
})

var hasil []int
hasil = append(hasil, ganjil...)
hasil = append(hasil, genap...)

for k, val := range hasil {
    if k > 0 {
        fmt.Print(" ")
    }
    fmt.Print(val)
}
fmt.Println()
}

```

Output :

The screenshot shows a Go IDE with the following components:

- EXPLORER:** Lists several files including `guided1.go`, `guided2.go`, `guided3.go`, `guided4.go`, `unguided1.go`, and `unguided2.go`.
- EDITOR:** Displays the source code for `unguided2.go`, which matches the code provided in the first block. The code is at line 51.
- TERMINAL:** Shows the command `go run unguided2.go` being executed. The output is:


```

5 2 1 7 9 13
1 7 9 13 2
6 189 27 15 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4

```

Penjelasan :

Program ini digunakan untuk mengelompokkan bilangan menjadi ganjil dan genap, kemudian mengurutkannya dengan aturan yang berbeda. Bilangan ganjil

diurutkan secara menaik (ascending), sedangkan bilangan genap diurutkan secara menurun (descending).

3. Soal Unguided 1 – Insertion Sort

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Jawaban :

```
package main

import (
    "fmt"
)

func main() {
    var input int
    var data []int

    for {
        fmt.Scan(&input)
        if input < 0 {
            break
        }
        data = append(data, input)
    }

    if len(data) == 0 {
        return
    }

    for i := 1; i < len(data); i++ {
        key := data[i]
        j := i - 1
        for j >= 0 && data[j] > key {
            data[j+1] = data[j]
```

```

        j--
    }
    data[j+1] = key
}

for i, val := range data {
    if i > 0 {
        fmt.Print(" ")
    }
    fmt.Print(val)
}
fmt.Println()

if len(data) < 2 {
    fmt.Println("Data berjarak 0")
    return
}

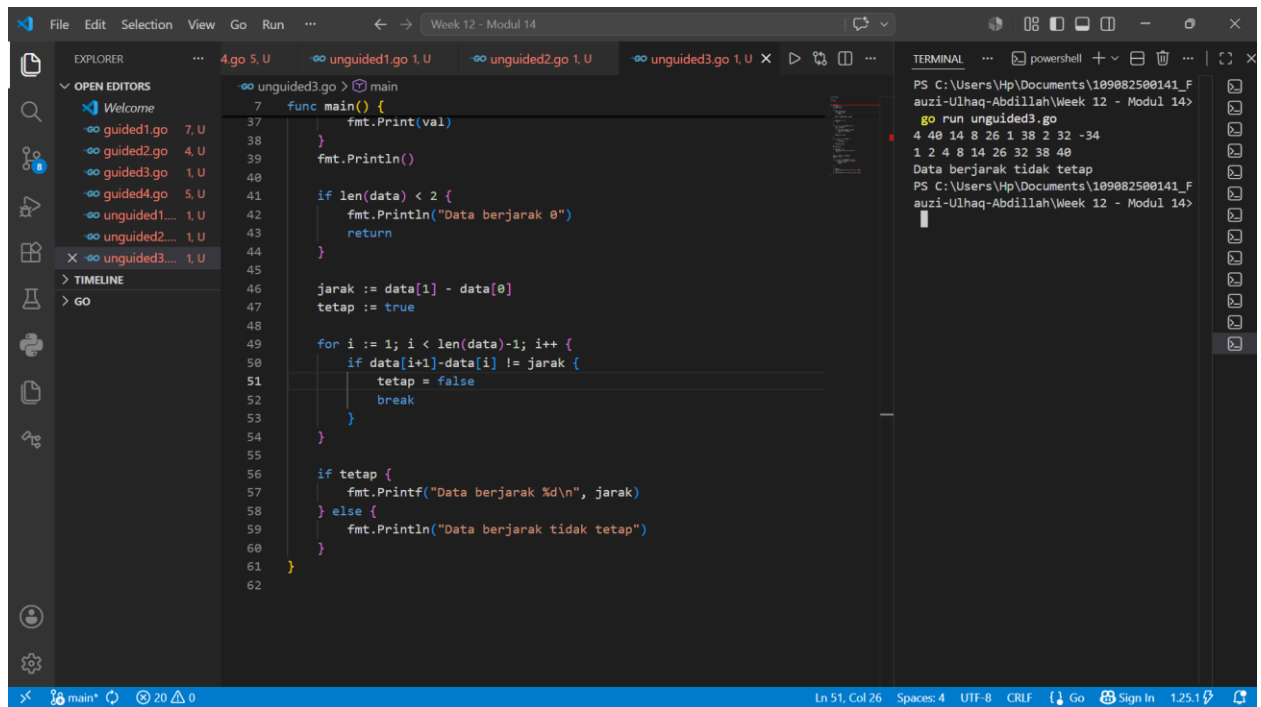
jarak := data[1] - data[0]
tetap := true

for i := 1; i < len(data)-1; i++ {
    if data[i+1]-data[i] != jarak {
        tetap = false
        break
    }
}

if tetap {
    fmt.Printf("Data berjarak %d\n", jarak)
} else {
    fmt.Println("Data berjarak tidak tetap")
}
}

```

Output :



```
File Edit Selection View Go Run ... Week 12 - Modul 14
EXPLORER
  OPEN EDITORS
    Welcome
    guided1.go 7, U
    guided2.go 4, U
    guided3.go 1, U
    guided4.go 5, U
    unguided1... 1, U
    unguided2... 1, U
    unguided3... 1, U
  TIMELINE
  GO
  unguided3.go main
    7 func main() {
    8     fmt.Print(val)
    9 }
   10 fmt.Println()
   11
   12 if len(data) < 2 {
   13     fmt.Println("Data berjarak 0")
   14     return
   15 }
   16
   17 jarak := data[1] - data[0]
   18 tetap := true
   19
   20 for i := 1; i < len(data)-1; i++ {
   21     if data[i+1]-data[i] != jarak {
   22         tetap = false
   23         break
   24     }
   25 }
   26
   27 if tetap {
   28     fmt.Printf("Data berjarak %d\n", jarak)
   29 } else {
   30     fmt.Println("Data berjarak tidak tetap")
   31 }
   32
   33 }
   34
   35
   36
   37
   38
   39
   40
   41
   42
   43
   44
   45
   46
   47
   48
   49
   50
   51
   52
   53
   54
   55
   56
   57
   58
   59
   60
   61
   62
TERMINAL
  powershell
  PS C:\Users\Hp\Documents\189082500141_F
  auzi-Ulhaq-Abdillah\Week 12 - Modul 14>
  go run unguided3.go
  4 40 14 8 26 1 38 2 32 -34
  1 2 4 8 14 26 32 38 40
  Data berjarak tidak tetap
  PS C:\Users\Hp\Documents\189082500141_F
  auzi-Ulhaq-Abdillah\Week 12 - Modul 14>
```

Penjelasan :

Kode Go-Lang diatas itu untuk membaca kumpulan bilangan bulat non-negatif yang diakhiri oleh bilangan negatif sebagai penanda berhenti (sentinel). Data tersebut kemudian diurutkan menggunakan metode Insertion Sort. Setelah terurut, program akan memeriksa selisih (jarak) antar elemen yang berdekatan untuk menentukan apakah polanya memiliki selisih yang konstan (tetap) atau tidak tetap.

4. Soal Unguided 2 – Insertion Sort

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan.

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Jawaban :

```
package main

import (
    "fmt"
)

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

const nMax = 7919

type DaftarBuku [nMax + 1]Buku

func main() {
    var Pustaka DaftarBuku
    var nPustaka int
    var ratingDicari int

    fmt.Scanln(&nPustaka)

    DaftarkanBuku(&Pustaka, nPustaka)

    CetakTerfavorit(Pustaka, nPustaka)

    UrutBuku(&Pustaka, nPustaka)

    Cetak5Terbaru(Pustaka, nPustaka)

    fmt.Scanln(&ratingDicari)

    CariBuku(Pustaka, nPustaka, ratingDicari)
}

func DaftarkanBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i <= n; i++ {
        fmt.Scanln(&pustaka[i].id)
        fmt.Scanln(&pustaka[i].judul)
        fmt.Scanln(&pustaka[i].penulis)
        fmt.Scanln(&pustaka[i].penerbit)
        fmt.Scanln(&pustaka[i].eksemplar)
        fmt.Scanln(&pustaka[i].tahun)
        fmt.Scanln(&pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
```

```

        return
    }
    idxMax := 1
    for i := 2; i <= n; i++ {
        if pustaka[i].rating > pustaka[idxMax].rating {
            idxMax = i
        }
    }
    b := pustaka[idxMax]
    fmt.Printf("%s, %s, %s, %d\n", b.judul, b.penulis,
b.penerbit, b.tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 2; i <= n; i++ {
        key := pustaka[i]
        j := i - 1
        for j >= 1 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }
        pustaka[j+1] = key
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    batas := 5
    if n < 5 {
        batas = n
    }
    for i := 1; i <= batas; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    low := 1
    high := n
    foundIdx := -1

    for low <= high {
        mid := (low + high) / 2
        if pustaka[mid].rating == r {
            foundIdx = mid
        } else if pustaka[mid].rating < r {
            high = mid - 1
        } else {
            low = mid + 1
        }
    }
}

```

```

        if foundIdx != -1 {
            b := pustaka[foundIdx]
            fmt.Printf("%s, %s, %s, %d, %d, %d\n", b.judul,
b.penulis, b.penerbit, b.tahun, b.eksemplar, b.rating)
        } else {
            fmt.Println("Tidak ada buku dengan rating seperti
itu")
        }
    }
}

```

Output :

The screenshot shows a Go IDE with the following components:

- EXPLORER:** Lists open files including `guided1.go` through `guided4.go` and `unguided1.go` through `unguided4.go`.
- EDITOR:** Displays the code for `unguided4.go`. The code includes functions `Cetak5Terbaru` and `CariBuku`. `CariBuku` implements a binary search algorithm to find a book by rating. The code snippet shown is:


```

74 func Cetak5Terbaru(pustaka DaftarBuku, n int) {
75 }
76
77 func CariBuku(pustaka DaftarBuku, n int, r int) {
78     low := 1
79     high := n
80     foundIdx := -1
81
82     for low <= high {
83         mid := (low + high) / 2
84         if pustaka[mid].rating == r {
85             foundIdx = mid
86         } else if pustaka[mid].rating < r {
87             high = mid - 1
88         } else {
89             low = mid + 1
90         }
91     }
92
93     if foundIdx != -1 {
94         b := pustaka[foundIdx]
95         fmt.Printf("%s, %s, %s, %d, %d, %d\n", b.judul, b.penulis, b.penerbit,
96         b.tahun, b.eksemplar, b.rating)
97     } else {
98         fmt.Println("Tidak ada buku dengan rating seperti itu")
99     }
100 }

```
- TERMINAL:** Shows the command `go run unguided4.go` and its output, which lists 15 books with their ratings:


```

PS C:\Users\Hp\Documents\18982500141_F
auzi-Ulhaq-Abdillah\Week 12 - Modul 14>
go run unguided4.go
95
B001
Bumi
Tere Liye
Gramedia
15
2014
90
B002
Negeri 5 Menara
Ahmad Fuadi
Gramedia
8
2009
88
B003
Ayat Ayat Cinta
Habiburrahman El Shirazy
Republika
12
2004
92
B004
Dilan 1990
Pidi Baiq
Pastel Books
20
2014
85
B005
Atomic Habits
James Clear
Avery

```

Penjelasan :

Kode Go-Lang diatas itu untuk mengelola data buku di perpustakaan menggunakan struktur data (struct dan array). Program ini melengkapi subprogram untuk mendaftarkan buku, mengurutkan data buku berdasarkan rating tertinggi secara menurun (descending) menggunakan Insertion Sort, menampilkan buku terfavorit, menampilkan 5 buku dengan rating tertinggi, serta mencari buku dengan rating tertentu menggunakan algoritma pencarian biner (Binary Search).

D. Kesimpulan Modul 14

Kesimpulan di atas berfokus pada penerapan struktur data (array dan struct). Gabungan dari seluruh soal diatas untuk melakukan pembacaan data (input handling) yang bervariasi, menerapkan algoritma pengurutan (Sorting) seperti Selection Sort dan Insertion Sort baik secara ascending maupun descending, melakukan analisis pola data (memeriksa jarak antar bilangan konstan atau memisahkan ganjil-genap), hingga melakukan pencarian data yang efisien (Searching) menggunakan metode Binary Search pada data yang sudah terurut.